

扫码签到系统

实现教程

Spring Boot + Redis + Hutool 实战项目

姓 名	谈天
班 级	软件2541
学 号	2505224130
技术栈	Spring Boot 3 · Redis · Hutool
核心功能	二维码生成 · 扫码签到 · 一次性使用
Redis类型	String (SET / GET / DEL)

有效期	10分钟自动过期
-----	----------

文档版本	v2.0 精排版
------	----------

2026年4月

目录

第一章

系统概述

1.1 项目背景 · 1.2 核心流程

第二章

技术架构

2.1 架构图 · 2.2 技术选型 · 2.3 项目结构

第三章

核心原理

3.1 生码 · 3.2 扫码 · 3.3 验码 · 3.4 一次性 · 3.5 防碰撞

第四章

UML建模

4.1 架构图 · 4.2 时序图 · 4.3 活动图 · 4.4 类图

第五章

代码详解

5.1 SignConstant · 5.2 QrSignService · 5.3 SignQrController

第六章

改进优化

6.1 安全性 · 6.2 功能扩展 · 6.3 性能优化

第七章

总结与答辩

7.1 核心知识 · 7.2 必问问题 · 7.3 满分话术

系统概述

1.1 项目背景与目标

本项目基于 Spring Boot 3 + Redis 实现一个课堂扫码签到系统。教师端生成二维码展示给学生，学生用手机微信扫一扫即可完成签到。整个系统不依赖数据库，所有数据通过 Redis 存储，充分体现 Redis 作为高速缓存的核心特性。

系统的三个核心目标：快速生码（秒级生成）、简单扫码（微信原生支持）、安全防护（一次性使用，截屏转发无效）。

1.2 核心流程总览

生码流程 — 教师访问 `/sign/createQr?activityId=xxx` → Controller 调用 Service → 生成 6 位大写随机码 → 防碰撞检查 → Redis 存储 (TTL=10分钟) → Hutool 生成二维码 → Base64 编码 → 返回 HTML 页面。

验码流程 — 学生微信扫描二维码 → 解析 URL 得到 code → 访问 `/sign/scan?code=xxx` → Service 从 Redis 查询 → 存在则删除 Key (一次性) → 返回签到成功页面；不存在则返回失效页面。

设计亮点

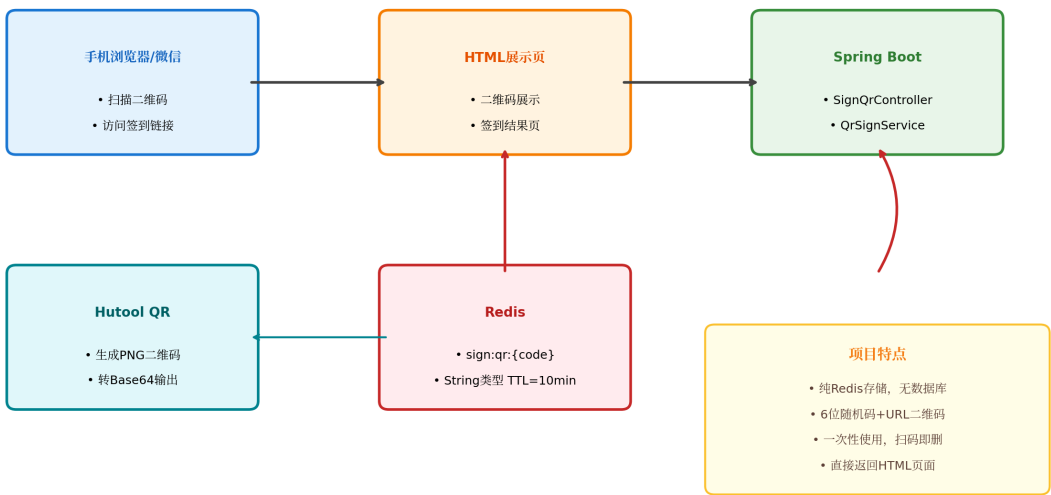
二维码内容是一个普通 HTTP URL，而非复杂加密 Token。这种设计的优点是任何支持扫码的浏览器或微信都可以直接使用，无需安装专门 App。缺点是 URL 中的 code 是明文，需要配合“一次性删除”机制保证安全。

技术架构设计

2.1 整体架构图

系统采用简洁的三层架构：表现层（Controller 返回 HTML）、业务层（Service 处理逻辑）、数据层（Redis 缓存）。没有引入数据库，所有数据通过 Redis 存储。

扫码签到系统架构图（实际项目）



2.2 技术选型说明

层级	组件	选型理由
表现层	Spring Boot + HTML内嵌	直接返回页面，无需前后端分离
业务层	Spring Service	Controller 接收请求，Service 处理业务
数据层	Redis String	SET/GET/DEL 满足全部需求，O(1)复杂度
工具库	Hutool	QrCodeUtil 和 RandomUtil 开箱即用

Redis客户端	StringRedisTemplate	Spring 官方推荐，专处理 String
----------	---------------------	------------------------

2.3 项目结构

实际项目文件结构如下：

```
src/main/java/top/mqxu/week08/scan/ ■■■ SignConstant.java # Redis
Key■■■■■■■■■■ ■■■ QrSignService.java # ■■■■■■■■ + ■■ ■■■
SignQrController.java # ■■■■■■■HTTP■■■ src/main/resources/ ■■■ application.yml #
Redis■■■■■
```

注意

项目中没有使用数据库（MySQL），所有数据完全依赖 Redis。这是为了体现 Redis 的高速缓存特性，但在生产环境中建议增加数据库做持久化备份。

核心原理详解

3.1 生码原理

生码流程由 `QrSignService.generateQrCode()` 方法实现，核心步骤如下：

① 生成6位大写随机码 — 使用 `Hutool` 的 `RandomUtil.randomStringUpper(6)`，包含 A-Z 共26个字母，理论组合数 $26 \approx 3.08$ 亿。

② 防碰撞检查 — 通过 `do-while` 循环检查 `Redis` 中是否已存在相同 code。存在则重新生成，确保全局唯一。

③ `Redis` 存储 — 使用 `StringRedisTemplate.opsForValue().set()`，Key 为 `sign:qr:{code}`，Value 为 `activityId`，TTL = 10分钟。

④ 生成二维码 — 拼接签到 URL，使用 `Hutool` `QrCodeUtil.generatePng()` 生成 300×300 像素 PNG。

⑤ Base64 编码 — 将 PNG 字节数组转为 Base64，前端通过 `data:image/png;base64`，前缀直接嵌入 `img` 标签。

3.2 扫码原理

二维码的内容是一个普通 HTTP URL，格式为 `http://服务器地址:8080/sign/scan?code=ABC123`。用户用微信扫一扫时，微信自动识别 URL 并跳转。

这种设计的优点是简单直观，任何支持扫码的浏览器或微信都可以使用，无需专门 App。缺点是 URL 中的 code 是明文，需要配合后续的一次性删除机制保证安全。

3.3 验码原理

验码流程由 `QrSignService.checkSignCode()` 实现，步骤如下：

① 参数校验 — `StringUtils.hasText(code)` 检查是否为空。

- ② Redis 查询 — `opsForValue().get("sign:qr:" + code)` 查询。
- ③ 有效性判断 — 返回 `null` 表示不存在或已过期，返回 `false`。
- ④ 删除 Key — 存在则立即 `delete(key)`，实现一次性使用。
- ⑤ 返回结果 — 删除成功返回 `true`，表示签到成功。

3.4 一次性使用机制

本项目最核心的安全设计就是一次性使用。第一个用户扫码成功后，Redis 中的 Key 被立即删除。第二个用户再次扫描同一个二维码时，Redis 已找不到该 Key，返回二维码已失效。

这比单纯设置 TTL 更进一步 —— 即使二维码在有效期内被截屏转发，也只能被第一个人使用。实现代码只有一行：`redisTemplate.delete(key)`，是 Redis 原子操作，并发场景下也安全。

3.5 防碰撞设计

6 位随机码虽然组合数很大，但高并发生成时仍可能碰撞。代码使用 `do-while` + `hasKey()` 防碰撞：

```
do { signCode = RandomUtil.randomStringUpper(6); } while  
(redisTemplate.hasKey(SignConstant.SIGN_CODE_KEY + signCode));
```

循环生成直到 Redis 中不存在该 Key。Redis 查询是 $O(1)$ 复杂度，即使有几次碰撞，整体耗时也在毫秒级。

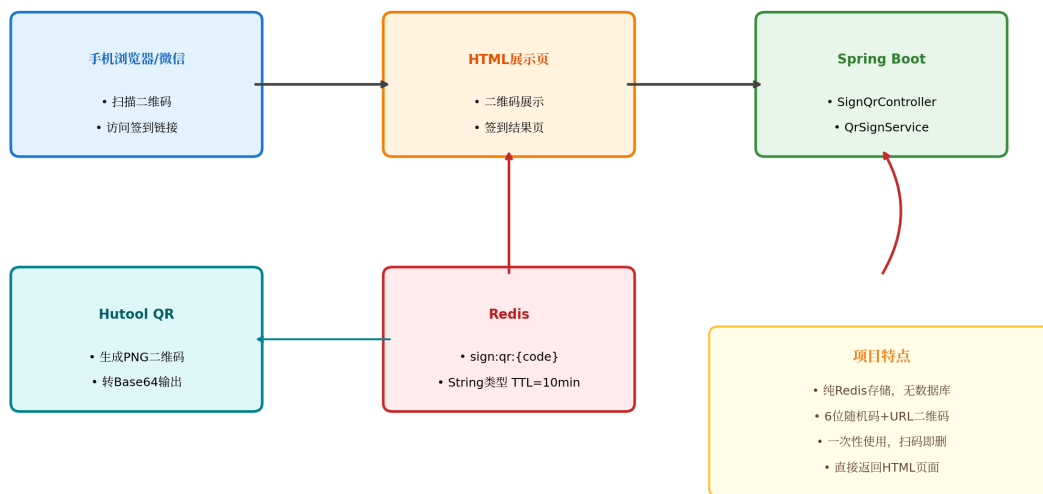
UML建模

本章通过 UML 图展示系统的静态结构和动态行为，所有图均基于实际代码绘制。

4.1 系统架构图

展示四层结构：手机浏览器/微信 → HTML展示页 → Spring Boot服务 → Redis，以及 Hutool QR 工具。

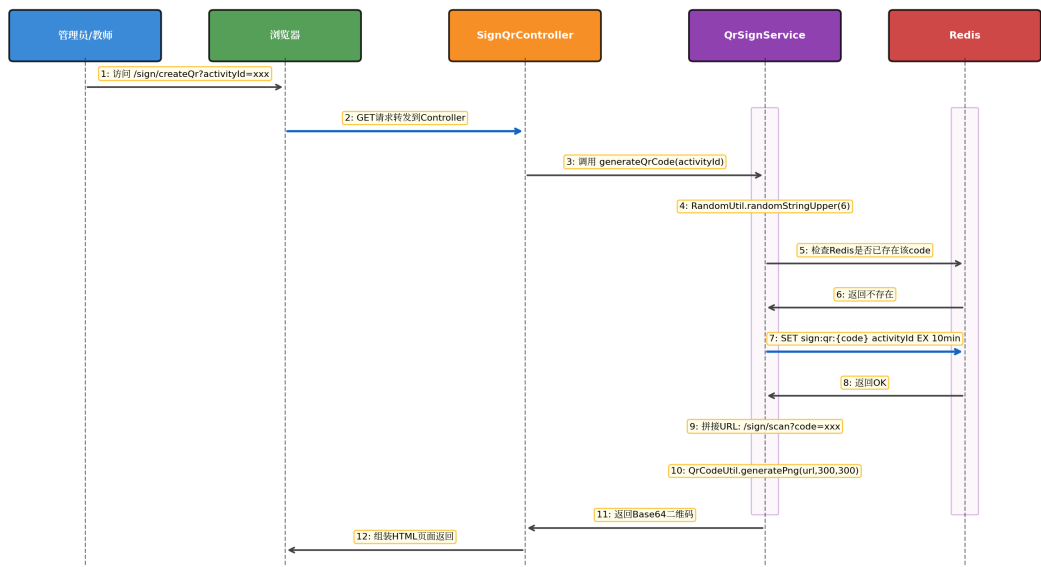
扫码签到系统架构图（实际项目）



4.2 生码时序图

从教师访问接口到浏览器展示二维码的完整交互。关键：生成随机码 → 防碰撞检查 → Redis SET → Hutool生码 → Base64 → HTML。

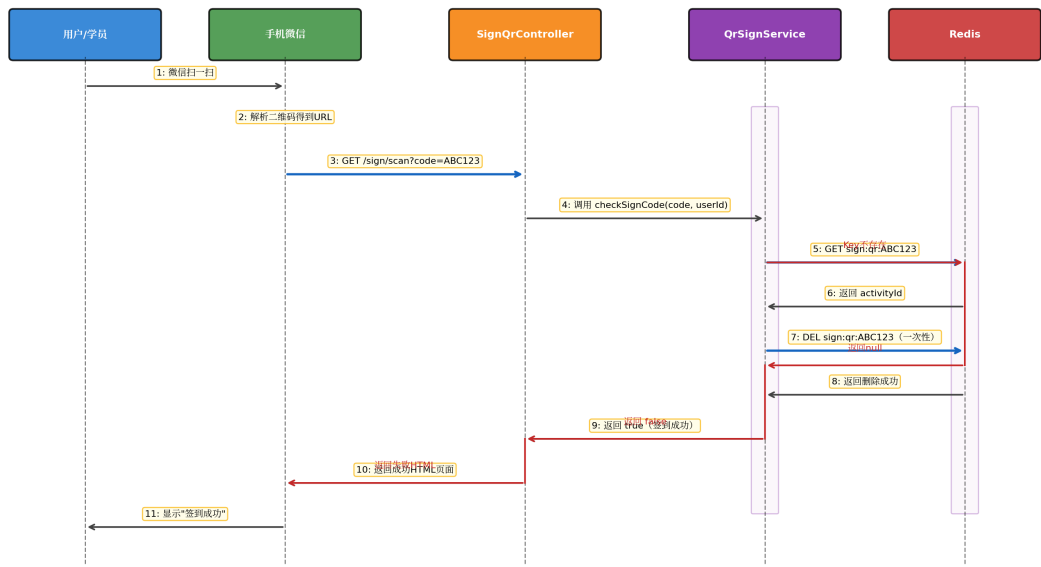
生成签到二维码时序图（实际代码流程）



4.3 扫码签到验码时序图

用户扫码后的处理流程。关键：解析URL获取code → GET请求 → Redis查询 → DEL删除（一次性）→ 返回成功/失败HTML。

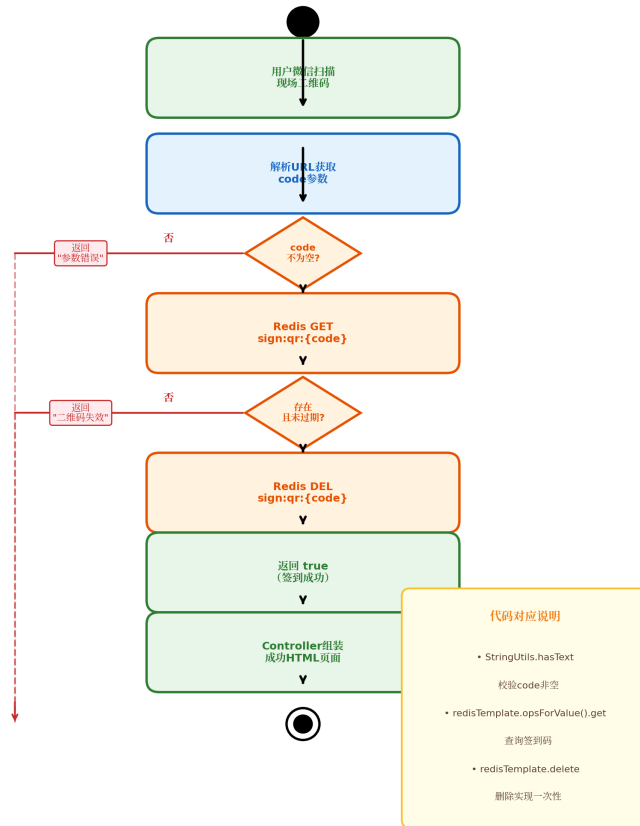
扫码签到验码时序图（实际代码流程）



4.4 活动图

扫码签到完整工作流，包含两个判断节点。右侧标注对应代码方法。

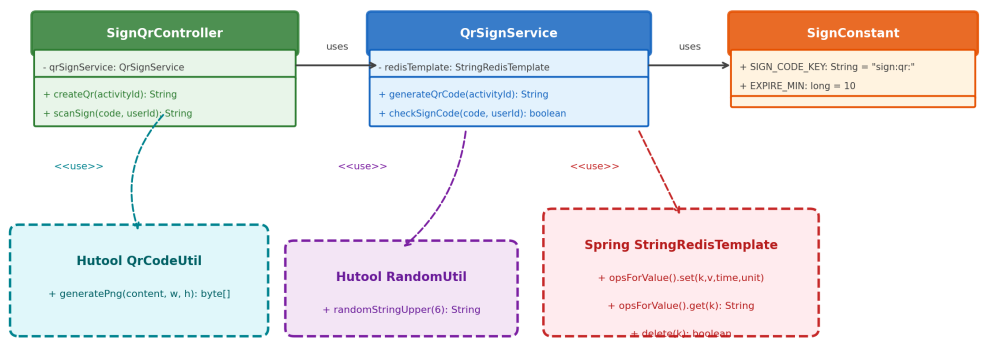
扫码签到活动图（实际代码流程）



4.5 类图

三个核心类 + 三个外部依赖。SignQrController 依赖 QrSignService ; QrSignService 依赖 SignConstant、StringRedisTemplate、QrCodeUtil、RandomUtil。

扫码签到系统类图（实际项目）



项目依赖关系说明

Spring Boot 提供 Web 容器和 RedisTemplate | Hutool 提供 QRCodeUtil 和 RandomUtil 工具类
SignQrController 接收 HTTP 请求并调用 QrSignService | QrSignService 操作 Redis 存储签到码
所有外部依赖使用 <<use>> 虚线箭头表示 | 内部依赖使用实线箭头表示

代码详解

本章逐行解析三个核心文件，说明每段代码的设计意图。

5.1 SignConstant.java

接口形式定义常量，集中管理 Redis Key 前缀和过期时间。

```
package top.mqxu.week08.scan; public interface SignConstant { // Redis
Key  sign:qr:  String SIGN_CODE_KEY = "sign:qr:"; //
10  long EXPIRE_MIN = 10; }
```

使用接口定义常量是 Java 惯用写法。Key 前缀统一便于通过 KEYS sign:qr:* 批量查看。过期时间 10 分钟是平衡点 —— 给足扫码时间，同时控制内存占用。

5.2 QrSignService.java

(1) 类声明与依赖注入

```
@Service @RequiredArgsConstructor public class QrSignService { private final
StringRedisTemplate redisTemplate; }
```

@RequiredArgsConstructor + final 字段是 Spring Boot 推荐的最佳实践，比 @Autowired 字段注入更利于单元测试。

(2) generateQrCode —— 生成二维码

```
public String generateQrCode(String activityId) { // 1. 6  String
signCode; do { signCode = RandomUtil.randomStringUpper(6); } while
(redisTemplate.hasKey( SignConstant.SIGN_CODE_KEY + signCode)); // 2.
RedisTTL=10  redisTemplate.opsForValue().set( SignConstant.SIGN_CODE_KEY +
signCode, activityId, SignConstant.EXPIRE_MIN, TimeUnit.MINUTES); // 3.
URL  String scanUrl =
"http://172.20.10.11:8080/sign/scan?code=" + signCode; // 4. Hutool PNG
byte[] bytes = QrCodeUtil.generatePng(scanUrl, 300, 300); // 5.
Base64  String base64 = Base64.getEncoder().encodeToString(bytes);
return "data:image/png;base64," + base64; }
```


5.3 SignQrController.java

(1) 类声明

```
@RestController @RequestMapping("/sign") @RequiredArgsConstructor public class SignQrController { private final QrSignService qrSignService; }
```

(2) createQr —— 生成并展示二维码

```
@GetMapping("/createQr") public String createQr(@RequestParam String activityId) { String qrBase64 = qrSignService.generateQrCode(activityId); // ■■■■HTML■■■■■■■■■■Base64■■■ return htmlTemplate.replace("PLACEHOLDER_QR", qrBase64); }
```

返回 String 而非 JSON，内容是一个完整 HTML 页面。页面使用现代 CSS（flex布局、渐变背景、圆角阴影）实现移动端友好的展示效果。二维码通过 `img` 标签的 `src="data:image/png;base64,xxx"` 直接嵌入。

(3) scanSign —— 扫码签到

```
@GetMapping("/scan") public String scanSign(@RequestParam String code, @RequestParam(defaultValue = "stu_1001") String userId) { boolean result = qrSignService.checkSignCode(code, userId); if (result) { return successHtml; // ■■■■ + ■■■■ } else { return failHtml; // ■■■■ + ■■■■ } }
```

`userId` 设置默认值 `stu_1001` 方便测试。根据 `boolean` 结果返回两种风格的 HTML：成功页面用绿色渐变 + 勾选动画，失败页面用红色渐变 + 抖动动画，视觉反馈明确。